



ΑΕΠΠ – Κεφάλαιο 1

Ανάλυση Προβλήματος

Αφίσα θεωρίας για αποστήθιση

Γ' Λυκείου

1.1 Η έννοια του προβλήματος



Πρόβλημα: μια κατάσταση που χρειάζεται αντιμετώπιση και λύση, ενώ η λύση της δεν είναι γνωστή ούτε προφανής.



- Τα προβλήματα εμφανίζονται στην καθημερινή ζωή και σε όλες τις επιστήμες.



- Η σωστή διατύπωση και η αναλυτική σκέψη είναι βασικά στοιχεία.



- Η ανάλυση προβλημάτων είναι χρήσιμο γνωστικό εργαλείο.

1.2 Κατανόηση προβλήματος



- Καμία προσπάθεια αντιμετώπισης δεν πετυχαίνει χωρίς πλήρη κατανόηση.
- Η κατανόηση εξαρτάται από δύο παράγοντες: σωστή διατύπωση και σωστή ερμηνεία.
- Ο λόγος πρέπει να χαρακτηρίζεται από σαφήνεια.
- Άστοχη ορολογία ή λανθασμένη σύνταξη μπορεί να οδηγήσει σε παραπλάνηση.

Κλασικό παράδειγμα ασάφειας

“ Ο Γιάννης και η Μαρία είναι παντρεμένοι. ”

Ερμηνεία 1:

Είναι παντρεμένοι μεταξύ τους.



Ερμηνεία 2:

Ο Γιάννης είναι παντρεμένος και η Μαρία είναι παντρεμένη.



★ Συμπέρασμα: η σωστή διατύπωση βοηθά στη σωστή κατανόηση.

3 Δεδομένα και Πληροφορίες



Δεδομένο

Κάθε στοιχείο που μπορεί να γίνει αντιληπτό από έναν τουλάχιστον παρατηρητή με μία από τις πέντε αισθήσεις.



Δεδομένα



Επεξεργασία



Πληροφορίες



Ο ανθρώπινος εγκέφαλος και ο υπολογιστής μπορούν να λειτουργήσουν ως μηχανισμοί επεξεργασίας δεδομένων.



Πληροφορία

Κάθε γνωστικό στοιχείο που προκύπτει από επεξεργασία δεδομένων.

82, 76, 91, 68,
88, 73, 95, 70

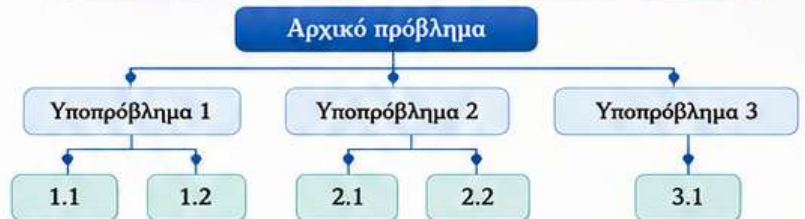
Μέσος όρος
= 80,4



Δομή προβλήματος: τα συστατικά μέρη του προβλήματος και ο τρόπος με τον οποίο συνδέονται μεταξύ τους.

- Η καταγραφή της δομής σημαίνει ότι αρχίζει η ανάλυση.
- Το πρόβλημα αναλύεται σε απλούστερα επιμέρους προβλήματα.
- Η δυσκολία μειώνεται όσο προχωρά η ανάλυση.
- Η διαδικασία σταματά όταν τα υποπροβλήματα είναι αρκετά απλά.

Διαγραμματική αναπαράσταση της δομής ενός προβλήματος



★ Στη διαγραμματική αναπαράσταση, το αρχικό πρόβλημα και τα απλούστερα προβλήματα παροσιάζονται σε διαφορετικά επίπεδα.

1 1.4 Καθορισμός απαιτήσεων

Ο καθορισμός απαιτήσεων ανήκει στο στάδιο της ανάλυσης.



Δεδομένα

- Τι παρέχει το πρόβλημα;
- Ποια στοιχεία είναι γνωστά;



Ζητούμενα

- Ποια αποτελέσματα πρέπει να παραχθούν;
- Με ποια μορφή θα παρουσιαστούν;



Διευκρινίσεις

- Μήπως κάποια δεδομένα πρέπει να εντοπιστούν μέσα στην εκφώνηση;
- Τι ακριβώς ζητά το πρόβλημα;
- Υπάρχουν απορίες για το εύρος ή τον τρόπο παρουσίασης των αποτελεσμάτων;

★ Η σωστή επίλυση προϋποθέτει ακριβή προσδιορισμό δεδομένων και ζητούμενων.

Βαθμολογίες μαθητών



→ ποσοστά επίδοσης



6 Τι να θυμάμαι για τις εξετάσεις;



- Πρόβλημα = κατάσταση που ζητά λύση.



- Πρώτα κατανοώ σωστά το πρόβλημα.



- Υστερα αναγνωρίζω τη δομή του.



- Καθορίζω με ακρίβεια δεδομένα και ζητούμενα.



- Τα δεδομένα, μετά από επεξεργασία, δίνουν πληροφορίες.



Κατανόηση • Ανάλυση • Επίλυση



ΑΝΑΛΥΣΗ ΠΡΟΒΛΗΜΑΤΟΣ



Π Ρ Ο Β Λ Η Μ Α



1. ΟΡΙΣΜΟΣ ΠΡΟΒΛΗΜΑΤΟΣ

Με τον όρο Πρόβλημα εννοείται μια κατάσταση η οποία χρήζει αντιμετώπισης, απαιτεί λύση, η δε λύση της δεν είναι γνωστή, ούτε προφανής.



2. ΚΑΤΑΝΟΗΣΗ ΠΡΟΒΛΗΜΑΤΟΣ

Η κατανόηση ενός προβλήματος αποτελεί συνάρτηση δύο παραγόντων, της σωστής διατύπωσης εκ μέρους του δημιουργού του και της αντίστοιχης σωστής ερμηνείας από τη μεριά εκείνου που καλείται να το αντιμετωπίσει.

Σαφήνεια διατύπωσης

Σαφήνεια διατύπωσης: Ο λόγος σαν μέσο επικοινωνίας και συνεννόησης πρέπει να χαρακτηρίζεται από σαφήνεια. Άστοχη χρήση ορολογίας, λανθασμένη σύνταξη, είναι δύο στοιχεία που μπορούν να προκαλέσουν παρερμηνείες και παραπληνίσεις.



ΣΧΕΣΕΙΣ ΒΑΣΙΚΩΝ ΕΝΝΟΙΩΝ

Πρόβλημα



Δεδομένο

Στοιχείο που δίνεται.



Ζητούμενα

Αποτελέσματα που πρέπει να βρεθούν.

Πληροφορία

Γνώση σχετικά που προκύπτει από την επεξεργασία δεδομένων.



ΠΡΟΒΛΗΜΑ



ΚΑΤΑΝΟΗΣΗ



ΑΝΑΛΥΣΗ



ΕΠΙΛΥΣΗ



3. ΔΕΔΟΜΕΝΑ ΚΑΙ ΠΛΗΡΟΦΟΡΙΑ



Δεδομένο: οποιοδήποτε στοιχείο μπορεί να γίνει αντιληπτό από έναν τουλάχιστον παρατηρητή με μία από τις πέντε αισθήσεις του.



Πληροφορία: οποιοδήποτε γνωστικό στοιχείο προέρχεται από επεξεργασία δεδομένων.



Επεξεργασία δεδομένων: διαδικασία κατά την οποία ένας μηχανισμός δέχεται δεδομένα, τα επεξεργάζεται με προκαθορισμένο τρόπο και αποδίδει πληροφορίες.



4. ΔΟΜΗ ΠΡΟΒΛΗΜΑΤΟΣ

Η καταγραφή της δομής ενός προβλήματος σημαίνει αναγνώριση ότι έχει αρχίσει η διαδικασία ανάλυσης του προβλήματος σε άλλα απλούστερα.

Με τον όρο δομή ενός προβλήματος αναφερόμαστε στα συστατικά του μέρη, στα επιμέρους τμήματα που το αποτελούν καθώς επίσης και στον τρόπο που αυτά τα μέρη συνδέονται μεταξύ τους.

ΠΑΡΑΔΕΙΓΜΑ ΔΟΜΗΣ ΠΡΟΒΛΗΜΑΤΟΣ



5. ΚΑΘΟΡΙΣΜΟΣ ΑΠΑΙΤΗΣΕΩΝ



Ακριβής προσδιορισμός δεδομένων



Λεπτομερειακά καταγραφή ζητούμενων



Διευκρίνιση τρόπου παρουσίασης αποτελεσμάτων



Προσοχή στην ανίχνευση δεδομένων και στην αποσαφήνιση των ζητούμενων



ΠΑΡΑΔΕΙΓΜΑ: ΤΟ ΠΡΟΒΛΗΜΑ ΤΟΥ ΤΑΧΥΔΡΟΜΟΥ



Στόχος: να επισκεφθεί όλες τις διευθύνσεις ακολουθώντας μια διαδρομή με συγκεκριμένα κριτήρια.

Απαιτείται κατανόηση, ανάλυση και σχεδιασμό λύσης.

ΣΤΑΔΙΑ ΑΝΤΙΜΕΤΩΠΙΣΗΣ ΠΡΟΒΛΗΜΑΤΟΣ



1 ΚΑΤΑΝΟΗΣΗ

Διευκρίνιση και πλήρης κατανόηση των δεδομένων και των ζητούμενων (τι μας δίνεται και τι ζητείται).



2 ΑΝΑΛΥΣΗ

Διάσπαση του προβλήματος σε απλούστερα υποπροβλήματα και μελέτη της μεταξύ τους συσχέτισης.



3 ΕΠΙΛΥΣΗ

Υλοποίηση της λύσης, εφαρμογή της μεθόδου και παρουσίαση των αποτελεσμάτων.



Στάδια αντιμετώπισης προβλήματος: κατανόηση → ανάλυση → επίλυση.



Κατανόηση = σωστή αποσαφήνιση δεδομένων και ζητούμενων.



Η σαφήνεια διατύπωσης είναι απαραίτητη για τη σωστή κατανόηση.



Δομή προβλήματος = συστατικά μέρη και τρόπος σύνδεσής τους.



Η ανάλυση διασπά το πρόβλημα σε απλούστερα υποπροβλήματα.

ΚΑΤΑΝΟΗΣΗ • ΑΝΑΛΥΣΗ • ΕΠΙΛΥΣΗ = Ο ΔΡΟΜΟΣ ΠΡΟΣ ΤΗ ΛΥΣΗ

ΚΕΦΑΛΑΙΟ 2

ΒΑΣΙΚΕΣ ΕΝΝΟΙΕΣ ΑΛΓΟΡΙΘΜΩΝ

Η ΤΕΧΝΗ ΤΗΣ ΣΥΣΤΗΜΑΤΙΚΗΣ ΕΠΙΛΥΣΗΣ

1. ΤΙ ΕΙΝΑΙ ΑΛΓΟΡΙΘΜΟΣ;

Αλγόριθμος είναι μια **πεπερασμένη σειρά οδηγιών** – εντολών, σαφών και εκτελέσιμων, που δίνουν λύση σε ένα πρόβλημα.

2. ΚΡΙΤΗΡΙΑ ΕΝΟΣ ΑΛΓΟΡΙΘΜΟΥ

- Είσοδος** Έχει 0 ή περισσότερα δεδομένα εισόδου.
- Έξοδος** Παράγει τουλάχιστον ένα αποτέλεσμα.
- Καθοριστικότητα** Κάθε εντολή είναι σαφής και χωρίς αμφισημία.
- Περατότητα** Ολοκληρώνεται μετά από πεπερασμένο αριθμό βημάτων.
- Αποτελεσματικότητα** Κάθε βήμα εκτελείται απλά και σε πεπερασμένο χρόνο.

3. ΟΙ 4 ΣΚΟΠΙΕΣ ΤΗΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΓΙΑ ΤΟΥΣ ΑΛΓΟΡΙΘΜΟΥΣ

- Υπολογιστική σκοπία** (Τι κάνει;) Ο αλγόριθμος λύνει ένα πρόβλημα.
- Γλωσσική σκοπία** (Πώς περιγράφεται;) Ο αλγόριθμος περιγράφεται με μια γλώσσα (φυσική, ψευδογλώσσα, γλώσσα προγραμματισμού).
- Εκτελεστική σκοπία** (Πώς εκτελείται;) Κάθε εντολή εκτελείται από τον υπολογιστή ή από τον άνθρωπο.
- Σκοπία απόδοσης** (Πόσο αποδοτικός είναι;) Μελέτη της επίδοσης: χρόνος εκτέλεσης και μνήμη που απαιτεί.

4. ΠΕΡΙΓΡΑΦΗ ΚΑΙ ΑΝΑΠΑΡΑΣΤΑΣΗ ΑΛΓΟΡΙΘΜΩΝ

Α. ΦΥΣΙΚΗ ΓΛΩΣΣΑ

Περιγραφή με λέξεις της καθημερινής γλώσσας.

- + Φυσική, κατανοητή
- Ασαφής, επιδέχεται παρανοήσεις

Β. ΨΕΥΔΟΓΛΩΣΣΑ

Μείγμα φυσικής γλώσσας και συμβολισμών.

- + Κατανοητή, ανεξάρτητη από γλώσσα προγραμματισμού
- Όχι τυπική, διαφορετικές παραλλαγές

Γ. ΔΙΑΓΡΑΜΜΑ ΡΟΗΣ

Γραφική αναπαράσταση με σύμβολα.

- + Οπτική, εύκολη επισκόπηση ροής
- Χρονοβόρα κατασκευή, δυσκολία σε μεγάλους αλγορίθμους

Δ. ΓΛΩΣΣΑ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ

Ακριβής περιγραφή σε συγκεκριμένη γλώσσα.

- + Ακριβής, εκτελέσιμη
- Εξαρτημένη από γλώσσα & περιβάλλον

ΣΥΜΠΕΡΑΣΜΑ

Καμία αναπαράσταση δεν είναι απόλυτα ιδανική. Επιλέγουμε ανάλογα με το στάδιο και τον σκοπό!

5. ΒΑΣΙΚΕΣ ΣΥΝΙΣΤΩΣΕΣ / ΕΝΤΟΛΕΣ ΕΝΟΣ ΑΛΓΟΡΙΘΜΟΥ

- Εντολή εκχώρησης** → Ανάθεση τιμής σε μεταβλητή.
- Εντολή εισόδου** → Διάβασμα τιμής από το περιβάλλον.
- Εντολή εξόδου** → Εκτύπωση ή εμφάνιση αποτελέσματος.
- Αριθμητικές πράξεις** +, -, *, /, div, mod, ^
- Λογικές (σχεσιακές) πράξεις** =, ≠, <, >, ≤, ≥
- Λογικές πράξεις** ΚΑΙ (and), Ή (or), ΟΧΙ (not)
- Δομές ελέγχου** → Ακολουθία, επιλογή, επανάληψη.

Σκέψου
Σχεδιάσε
Υλοποίησε
Βελτίωσε!

6. ΣΥΜΒΟΛΑ ΔΙΑΓΡΑΜΜΑΤΟΣ ΡΟΗΣ

Σύμβολο	Ονομασία	Χρήση
	Αρχή / Τέλος	Έναρξη ή τερματισμός αλγορίθμου
	Είσοδος / Έξοδος	Διάβασμα δεδομένων ή εμφάνιση αποτελεσμάτων
	Επεξεργασία	Εκτέλεση ενέργειας ή υπολογισμού
	Απόφαση	Έλεγχος συνθήκης - διακλάδωση
	Ροή	Κατεύθυνση εκτέλεσης

ΣΗΜΕΙΩΣΗ

Δεν είναι υποχρεωτικό να χρησιμοποιούμε διάγραμμα ροής!

7. ΨΕΥΔΟΓΛΩΣΣΑ (ΠΑΡΑΔΕΙΓΜΑ)

Αλγόριθμος Παράδειγμα

Διάβασε a, b
c ← a + b
Εκτύπωσε c
Τέλος Παράδειγμα

Η ψευδογλώσσα:

- ✓ Δεν είναι γλώσσα προγραμματισμού.
- ✓ Χρησιμοποιεί απλές ελληνικές λέξεις.
- ✓ Κατανοητή από όλους.
- ✓ Βοηθά στο σχεδιασμό του αλγορίθμου.

8. ΠΟΛΛΑΠΛΑΣΙΑΣΜΟΣ ΑΛΛΑ ΡΩΣΙΚΑ (Παράδειγμα)

Για 15×13 με τη μέθοδο του διπλασιασμού και υποδιπλασιασμού (ρωσικός πολλαπλασιασμός):

- Διπλασιάζουμε τον πρώτο αριθμό (α) και υποδιπλασιάζουμε τον δεύτερο (β).
- Αν ο β είναι περιττός, προσθέτουμε τον αντίστοιχο α στο άθροισμα.
- Επαναλαμβάνουμε μέχρι ο β να γίνει 0.

$$15 \times 13 = 195$$

α (διπλασιασμός)	β (υποδιπλασιασμός)	Απόφαση	Άθροισμα
15	13	περιττός → +15	15
30	6	άρτιος	15
60	3	περιττός → +60	75
120	1	περιττός → +120	195
240	0	Τέλος	195

★ Ίδια ιδέα με τις δομές: επανάληψη + επιλογή!

Ο ΑΛΓΟΡΙΘΜΟΣ ΕΙΝΑΙ Η ΓΕΦΥΡΑ ΑΠΟ ΤΗ ΣΚΕΨΗ ΣΤΗΝ ΥΛΟΠΟΙΗΣΗ!



ΚΕΦΑΛΑΙΟ 3 — ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ ΚΑΙ ΑΛΓΟΡΙΘΜΟΙ

Αποστήθιση Θεωρίας | Χωρίς Αναδρομή



3.1 ΔΕΔΟΜΕΝΑ

- Τα δεδομένα είναι η αφαιρετική αναπαράσταση της πραγματικότητας, δηλαδή καταγεγραμμένη γεγονότα.
- Η συλλογή και η συστηματικός δεδομένων δίνει πληροφορία.
- Ο αλγόριθμος είναι το μέσον για την παραγωγή πληροφορίας από τα δεδομένα.

Αρχείο μαθητών: χρήσιμα δεδομένα είναι ονοματεπώνυμο, ηλικία, φύλο, τάξη, τμήμα.

Η Πληροφορία μάλαει τα δεδομένα από 4 σκοπές:



3.2 ΑΛΓΟΡΙΘΜΟΙ + ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ = ΠΡΟΓΡΑΜΜΑΤΑ

Αλγόριθμοι + Δομές Δεδομένων = Προγράμματα

Δομές Δεδομένων είναι ένα σύνολο αποθηκευμένων δεδομένων που υφίστανται επεξεργασία από ένα σύνολο λειτουργιών.

Το πρόγραμμα θεμελιώνει τη δομή δεδομένων και τον αλγόριθμο ως μια διαδικατική ενότητα.

- Στατικές:** τα ακριβές μέγεθος της μνήμης καθορίζεται κατά τη στιγμή του προγραμματισμού/επιτοίχισης, και το στοιχείο αποθηκεύονται σε συνεχόμενες θέσεις μνήμης.
- Δυναμικές:** δεν αποθηκεύονται σε συνεχόμενες θέσεις μνήμης, επιρρίζονται από δυναμική παραχώρηση μνήμης και το μέγεθός τους μεταβάλλεται.

Στο βιβλίο στη συνέχεια εξετάζονται οι στατικές δομές.



ΟΙ 8 ΒΑΣΙΚΕΣ ΛΕΙΤΟΥΡΓΙΕΣ ΕΠΙ ΤΩΝ ΔΟΜΩΝ ΔΕΔΟΜΕΝΩΝ



Στην πράξη συνήθως η ίδια δομή δεν χρησιμοποιείται εξίσου αποδοτικά για όλες τις λειτουργίες.

3.3 ΠΙΝΑΚΕΣ

- Οι πίνακες υλοποιούν στατικές δομές και περιέχουν στοιχεία του ίδιου τύπου.
- Η αναφορά στα στοιχεία ενός πίνακα γίνεται με το συμβολικό όνομα και έναν ή περισσότερους δείκτες.
- Ένας πίνακας μπορεί να είναι μονοδιάστατος, διδιάστατος, τριδιάστατος ή n-διάστατος.
- Διδιάστατος: πίνακας με ίσες διαστάσεις είναι τετράγωνος ($n \times n$).

Μονοδιάστατος πίνακας (1D)
table[1..5]

7	14	21	28	35
---	----	----	----	----

Διδιάστατος πίνακας (2D)
table[1..3, 1..4]

	1	2	3	4
1	4	8	12	16
2	5	10	15	20
3	6	12	18	24

Τριδιάστατος πίνακας (3D)
table[1..2, 1..2, 1..3]

i=1	1	2	3
	4	5	6
i=2	7	8	9
	10	11	12



Παράδειγμα 1: Εύρεση του μικρότερου στοιχείου ενός μονοδιάστατου πίνακα table 100 στοιχείων.

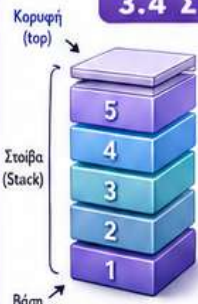
table[1..9]

52	12	71	56	5	10	19	90	45
----	----	----	----	---	----	----	----	----



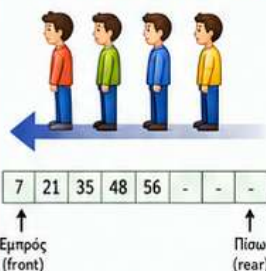
Παράδειγμα 2: Διδιάστατος πίνακας table με m γραμμές και n στήλες – άθροισμα κατά γραμμή, κατά στήλη και συνολικά.

3.4 ΣΤΟΙΒΑ



- Λειτουργεί με τη μέθοδο LIFO (Last-In-First-Out).
- Κύριες λειτουργίες: ώθηση (push), απώθηση (pop).
- Κορυφή (top).
- Έλεγχος υπερχείλισης (overflow) και υποχειλίσας (underflow).
- Υλοποίηση με μονοδιάστατο πίνακα και βοηθητική μεταβλητή top.

3.5 ΟΥΡΑ



- Λειτουργεί με τη μέθοδο FIFO (First-In-First-Out).
- Κύριες λειτουργίες: εισαγωγή (enqueue), εξαγωγή (dequeue).
- Δύο δείκτες: εμπρός (front) και πίσω (rear).
- Υλοποιείται με μονοδιάστατο πίνακα.
- Πριν από κάθε εισαγωγή ελέγχουμε αν υπάρχει διαθέσιμη θέση πριν από κάθε εξαγωγή ελέγχουμε αν η ουρά είναι άδεια.

3.6 ΑΝΑΖΗΤΗΣΗ

- Η πιο απλή μορφή είναι η σειριακή αναζήτηση.
- Γίνεται σε έναν πίνακα.
- Σε επιτυχία η θέση μνήμης ευρίσκει από 1 έως n, ενώ σε αποτυχία επιστρέφεται 0.

- $i = 0$, μέθοδος διαδοχικών ισών άσων:
- ο πίνακας είναι μη ταξινομημένος
- ο πίνακας είναι με μοναδικές τιμές (π.χ. $n \leq 20$)
- η αναζήτηση σε έναν ευρετηριασμένο πίνακα γίνεται σε λίγα βήματα.

Μη ταξινομημένος πίνακας:

52	12	71	56	5	10	19	90	45
----	----	----	----	---	----	----	----	----

Αναζήτηση 56: τερματίζεται με 4 προσπελάσεις.

Αναζήτηση 13: δεν υπάρχει στον πίνακα → 9 προσπελάσεις (αποτυχία, θέση επιστροφής 0).

Ταξινομημένος πίνακας:

5	10	12	19	45	52	56	71	90
---	----	----	----	----	----	----	----	----

Αναζήτηση 11: επιτυχία στο 3^ο βήμα (ανάγνωση του 12).

3.7 ΤΑΞΙΝΟΜΗΣΗ

- Η ταξινόμηση είναι η τακτοποίηση των δεδομένων μίας δομής σε διάταξη μεταξύ, συνήθως αύξουσα.
- Σκοπός της είναι να διευκολύνει η αναζήτηση.
- Με αναζήτηση διατάξεις f:

$$f(a_1) \leq f(a_2) \leq \dots \leq f(a_n)$$

- Η ταξινόμηση μπορεί να είναι και φθίνουσα.

Ευθεία ανταλλαγή (straight exchange sort) / Φουσάλιδα (bubble sort): Βρίσκουμε τη θέση στη τελευταία μεγαλύτερη στοιχείο και την ανταλλάσσουμε με το στοιχείο που βρίσκεται εκεί.

Παράδειγμα:

A:	5	1	4	2	8
Πέρασμα 1:	1	5	4	8	2
Πέρασμα 2:	1	1	5	2	8
Πέρασμα 3:	1	4	2	5	8

(Τέλος)

ΆΛΛΕΣ ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ

- Λίστα
- Δέντρο
- Δομές διασυνδεδεμένου γραφικού

Αρχεία (files): Τα στοιχεία ενός αρχείου γράφονται σε εγγραφές (records). Κάθε εγγραφή αποτελείται από πεδία (fields), και υπάρχουν πεπονημένα διαφορετικοί τύποι.



Οι πίνακες και οι άλλες δομές βασικές δομές κύριων μνήμης χάνουν τα δεδομένα τους όταν το πρόγραμμα σταματήσει. Τα αρχεία διατηρούνται.

ΤΕΛΙΚΗ ΣΥΝΟΨΗ ΓΙΑ ΜΝΗΜΗ!

1. Δεδομένα = Πληροφορία

2. Αλγόριθμος = Δομές Δεδομένων = Προγράμματα

3. 8 λειτουργίες στις δομές δεδομένων

4. Πίνακες: στατικές δομές ίδιου τύπου

5. Στοιβά = LIFO (ώθηση, απώθηση)

6. Ουρά = FIFO (εισαγωγή, εξαγωγή)

7. Σειριακή αναζήτηση

8. Ταξινόμηση

9. Λίστα, δέντρο, αρχείο, δομές διασυνδεδεμένου



ΑΝΑΛΥΣΗ ΠΡΟΒΛΗΜΑΤΟΣ

Μέθοδοι ανάλυσης και επίλυσης προβλημάτων



1 ΕΙΣΑΓΩΓΗ – Η ΙΔΕΑ



Πριν από την κωδικοποίηση και τον προγραμματισμό, προηγείται η θεωρητική μελέτη και ανάλυση του προβλήματος.

Ένα πρόβλημα μπορεί να έχει περισσότερες από μία λύσεις.



Η σωστή ανάλυση είναι απαραίτητη, ώστε να προτείνουμε συγκεκριμένη μεθοδολογία και ακολουθία βημάτων.

Βασικός μας στόχος:
να προτείνουμε έξυπνες και αποδοτικές λύσεις!

2 Η ΑΝΑΛΥΣΗ ΕΝΟΣ ΠΡΟΒΛΗΜΑΤΟΣ ΠΕΡΙΛΑΜΒΑΝΕΙ

- 1 Καταγραφή της υπάρχουσας πληροφορίας για το πρόβλημα.
- 2 Αναγνώριση των ιδιαιτεροτήτων του προβλήματος.
- 3 Αποτύπωση των συνθηκών και προϋποθέσεων υλοποίησης.
- 4 Πρόταση επίλυσης με χρήση κάποιας μεθόδου.
- 5 Τελική επίλυση με χρήση υπολογιστικών συστημάτων.

3 ΕΡΩΤΗΜΑΤΑ ΠΟΥ ΠΡΕΠΕΙ ΝΑ ΑΠΑΝΤΗΘΟΥΝ

- ? Ποια είναι τα δεδομένα και το μέγεθος του προβλήματος;
- ? Ποιες συνθήκες πρέπει να πληρούνται;
- ? Ποια είναι η πιο αποδοτική μέθοδος επίλυσης;
- ? Πώς θα καταγραφεί η λύση; (π.χ. ψευδογλώσσα)
- ? Πώς θα υλοποιηθεί στο υπολογιστικό σύστημα; (π.χ. επιλογή γλώσσας)

4 ΠΑΡΑΔΕΙΓΜΑ: ΤΟ ΠΡΟΒΛΗΜΑ ΤΟΥ ΤΑΧΥΔΡΟΜΟΥ



Ένας ταχυδρόμος ξεκινά από το χωριό 1, πρέπει να επισκεφθεί τα χωριά 2, 3, 4 και να επιστρέψει στο χωριό 1, περνώντας από κάθε χωριό μόνο μία φορά.

Στόχος του να διανύσει όσες το δυνατόν λιγότερες συνολικά χιλιόμετρα.



4Α ΠΡΩΤΗ ΑΝΑΛΥΣΗ

- 1 Καταγραφή όλων των αποστάσεων.
- 2 Ταξινόμηση των συνδέσεων κατά αύξουσα απόσταση.
- 3 Κάθε φορά μετάβαση στο πλησιέστερο χωριό.

Σειρά επίσκεψης:

1 → 2 → 4 → 3 → 1

Συνολική απόσταση:

36 χιλιόμετρα

4Β ΔΙΑΦΟΡΕΤΙΚΗ ΑΝΑΛΥΣΗ

- 1 Καταγραφή όλων των αποστάσεων.
- 2 Εύρεση σειράς επίσκεψης με στόχο την ελαχιστοποίηση της συνολικής απόστασης, όχι της κάθε φορά απόστασης.

Σειρά επίσκεψης:

1 → 4 → 3 → 2 → 1

Συνολική απόσταση:

30 χιλιόμετρα

Η καλύτερη διαδρομή!

5 ΤΙ ΜΑΣ ΔΙΔΑΣΚΕΙ ΤΟ ΠΑΡΑΔΕΙΓΜΑ



Η ανάλυση είναι απαραίτητη για να βρεθεί η καταλληλότερη μέθοδος.



Στόχος είναι λύση όσο γίνεται ταχύτερη και με το μικρότερο κόστος σε υπολογιστικούς πόρους.



Δεν υπάρχει μία ενιαία φόρμουλα για όλα τα προβλήματα.



Υπάρχουν όμοια «συγγενή» προβλήματα που αντιμετωπίζονται με παρόμοιες τεχνικές.

6 ΓΙΑΤΙ ΕΧΟΥΝ ΕΝΔΙΑΦΕΡΟΝ ΟΙ ΜΕΘΟΔΟΙ ΑΝΑΛΥΣΗΣ ΚΑΙ ΕΠΙΛΥΣΗΣ;



Παρέχουν ένα γενικό πρότυπο για προβλήματα ευρείας κλίμακας.



Μπορούν να αναπαρασταθούν με κοινές δομές δεδομένων και ελέγχου, που υποστηρίζονται από τις περισσότερες σύγχρονες γλώσσες προγραμματισμού.



Επιτρέπουν την καταγραφή των χρονικών και χωρικών απαιτήσεων, ώστε να γίνεται ακριβής εκτίμηση των αποτελεσμάτων.



ΚΡΑΤΗΣΕ ΓΙΑ ΤΗ ΜΝΗΜΗ



Πρώτα αναλύουμε, μετά προγραμματίζουμε.



Το ίδιο πρόβλημα μπορεί να έχει πολλές λύσεις.



Η καλύτερη λύση δεν είναι πάντα η προφανής.



Ζητούμενο: αποδοτική λύση με μικρό κόστος πόρων.



Δεν υπάρχει μία μέθοδος για όλα τα προβλήματα.

1 Η έννοια του προγράμματος

Η επίλυση προβλήματος με υπολογιστή περιλαμβάνει 3 στάδια:

1 Ακριβής προσδιορισμός του προβλήματος

2 Ανάπτυξη αλγορίθμου

3 Διατύπωση του αλγορίθμου σε μορφή κατανοητή από τον υπολογιστή

**Πρόγραμμα:**
σύνολο εντολών που δίνεται στον υπολογιστή για την υλοποίηση ενός αλγορίθμου.

**Το πρόγραμμα δεν είναι μόνο οι εντολές:**
περιλαμβάνει και τα δεδομένα / δομές δεδομένων πάνω στα οποία ενεργεί.

**Ο υπολογιστής καταλαβαίνει τελικά μόνο 0 και 1**
(δυναδικό σύστημα).

2 Φυσικές και τεχνητές γλώσσες

Κάθε γλώσσα προσδιορίζεται από:
αλφάβητο, λεξιλόγιο, γραμματική, σημασιολογία.

1 ΑΩ

Αλφάβητο
Το σύνολο των στοιχείων της γλώσσας.
Παράδειγμα:
Στην Pascal: γράμματα, ψηφία, ειδικοί χαρακτήρες.

2 abc

Λεξιλόγιο
Οι αποδεκτές λέξεις που σχηματίζονται από το αλφάβητο.
Παράδειγμα:
sum, total, x1 είναι δεκτές λέξεις - @abc δεν είναι.

3

Τυπικό (τυπολογικό)
Οι μορφές με τις οποίες μία λέξη είναι αποδεκτή.
Παράδειγμα:
count είναι σωστό αναγνωριστικό, 2count δεν είναι.

4

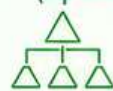
Συντακτικό
Οι κανόνες διάταξης και σύνδεσης των λέξεων για σωστές εντολές.
Παράδειγμα:
AN x>0 ΤΟΤΕ ΓΡΑΨΕ x ✓
ΤΟΤΕ x>0 AN ΓΡΑΨΕ x ✗

5

Σημασιολογία
Το νόημα μιας λέξης ή εντολής.
Παράδειγμα:
x ← x + 1
σημαίνει: αύξησε την τιμή της x κατά 1.

3 Τεχνικές σχεδίασης προγραμμάτων

A Ιεραρχική σχεδίαση (top-down)



- Διάσπαση του αρχικού προβλήματος σε απλούστερα υποπροβλήματα.
- Ξεκινάμε από τις βασικές λειτουργίες και προχωρούμε σε ολοένα μικρότερες.

B Τμηματικός προγραμματισμός



- Κάθε υποπρόβλημα γίνεται ανεξάρτητη ενότητα (module).
- Διευκολύνει δημιουργία, κατανόηση, συντήρηση και μείωση λαθών.

Γ Δομημένος προγραμματισμός



- Βασίζεται σε 3 λογικές δομές: ακολουθία, επιλογή, επανάληψη.
- Κάθε πρόγραμμα / τμήμα έχει μία είσοδο και μία έξοδο.
- Στόχος: λιγότερα λάθη, καλύτερη κατανόηση, ευκολότερη συντήρηση.

Πλεονεκτήματα δομημένου προγραμματισμού

**Απλούστερα προγράμματα**

**Άμεση μεταφορά αλγορίθμων σε προγράμματα**

**Περιορισμός λαθών**

**Ευκολότερη ανάγνωση, διόρθωση και συντήρηση**

Η εντολή GOTO θεωρήθηκε επιβλαβής γιατί δυσκολεύει την κατανόηση της ροής.

4 Αντικειμενοστραφής προγραμματισμός

Η έμφαση μεταφέρεται από τις ενέργειες στα δεδομένα.

Βασικά δομικά στοιχεία: αντικείμενα (objects).

Πλεονεκτήματα: μεγαλύτερη ευελιξία, καλύτερη οργάνωση, επαναχρησιμοποίηση.

Ακολουθεί επίσης τις αρχές ιεραρχικής σχεδίασης, τμηματικού και δομημένου προγραμματισμού.

Η έννοια σε 3 βήματα



Αντικείμενο = δεδομένα + λειτουργίες που τα επεξεργάζονται.

5 Προγραμματιστικά περιβάλλοντα

Κάθε πρόγραμμα πρέπει να μετατραπεί σε γλώσσα μηχανής για να εκτελεστεί.

**Πηγαίο πρόγραμμα (source)**
Το αρχικό πρόγραμμα του προγραμματιστή.

**Αντικείμενο πρόγραμμα (object)**
Το αποτέλεσμα της μεταγλώττισης.

**Εκτελέσιμο πρόγραμμα (executable)**
Το τελικό πρόγραμμα που εκτελείται.

**Συνδέτης-φορτωτής (linker-loader)**
Συνδέει το αντικείμενο πρόγραμμα με τα απαραίτητα τμήματα / βιβλιοθήκες.

**Συντακτικά λάθη:**
εντοπίζονται στη μεταγλώττιση.

**Λογικά λάθη:**
εμφανίζονται στην εκτέλεση.

6 Γραφική αναπαράσταση της διαδικασίας μεταγλώττισης



Κύκλος εντοπισμού και διόρθωσης λαθών



7 Μεταγλωττιστής vs Διερμηνευτής

Μεταγλωττιστής (Compiler)



Μεταφράζει ολόκληρο το πρόγραμμα σε γλώσσα μηχανής.

- ✓ Παράγει αντικείμενο / εκτελέσιμο πρόγραμμα.
- ✓ Η εκτέλεση του τελικού προγράμματος είναι συνήθως ταχύτερη.

Παράδειγμα: Μεταγλωττίζω όλο το πρόγραμμα και μετά το εκτελώ.

Διερμηνευτής (Interpreter)



Μεταφράζει και εκτελεί μία-μία τις εντολές.

- ✓ Δεν παράγει ανεξάρτητο εκτελέσιμο πρόγραμμα με τον ίδιο τρόπο.
- ✓ Διευκολύνει την άμεση δοκιμή και διόρθωση, αλλά η εκτέλεση είναι συχνά πιο αργή.

Παράδειγμα: Εκτελώ κάθε εντολή αμέσως μόλις τη διαβάσει ο διερμηνευτής.

**Compiler:** μεταφράζω ολόκληρο βιβλίο πριν το διαβάσω.

VS

**Interpreter:** μεταφράζω κάθε πρόταση τη στιγμή που τη διαβάζω.

8 Τι περιέχει ένα σύγχρονο προγραμματιστικό περιβάλλον;

**Συντάκτης (editor)**
Γράφουμε και επεξεργαζόμαστε τον κώδικα του προγράμματος.

**Μεταγλωττιστή ή διερμηνευτή**
Μεταφράζει τον κώδικα και ελέγχει τα συντακτικά λάθη.

**Συνδέτης**
Συνδέει το πρόγραμμα με βιβλιοθήκες και άλλα τμήματα.

**Εργαλεία εκτέλεσης και διόρθωσης**
Εκτελούμε, ελέγχουμε, εντοπίζουμε και διορθώνουμε λάθη.

Τα σύγχρονα ολοκληρωμένα περιβάλλοντα παρέχουν ενιαία όλα τα απαραίτητα εργαλεία για συγγραφή, μετάφραση, εκτέλεση και διόρθωση προγραμμάτων.

1 ΑΛΦΑΒΗΤΟ

Σύνολο συμβόλων που χρησιμοποιεί μια γλώσσα.

α	Πεζά ελληνικά γράμματα	α-ω
A	Κεφαλαία λατινικά γράμματα	A-Z
a	Πεζά λατινικά γράμματα	a-z
9	Ψηφία	0-9
{..}	Ειδικοί χαρακτήρες	+ - = / < > () , κενό: & κενός χαρακτήρας ^

2 ΤΥΠΟΙ ΔΕΔΟΜΕΝΩΝ

Καθορίζουν το είδος των τιμών που μπορούν να πάρουν τα δεδομένα.

Τύπος	Περιγραφή	Παραδείγματα
Ακέραιος τύπος	Περιλαμβάνει ακέραιους που είναι φυσικοί αριθμοί μαθηματικά.	1, 3409, -980
Πραγματικός τύπος	Περιλαμβάνει τους πραγματικούς αριθμούς που γράφουμε από τα μαθηματικά.	3,14159, 2,71828, 112,45, 0.0, -5.6
Χαρακτήρας	Αναφέρεται σε έναν χαρακτήρα.	'A', 'x', '5', ','
Λογικός τύπος	Δέχεται μόνο δύο τιμές: ΑΛΗΘΗΣ ή ΨΕΥΔΗΣ.	ΑΛΗΘΗΣ, ΨΕΥΔΗΣ

3 ΣΤΑΘΕΡΕΣ

Προσκαθορισμένες τιμές που δεν μεταβάλλονται κατά τη διάρκεια εκτέλεσης του προγράμματος.

Συμβολικές σταθερές

Η ΓΛΩΣΣΑ επιτρέπει την αντικατάσταση σταθερών τιμών με ονόματα, εφόσον αυτά δηλωθούν στην αρχή του προγράμματος.

Παράδειγμα:

```
ΣΤΑΘΕΡΕΣ  
ΠΟΣΟΣΤΟ = 24  
ΟΝΟΜΑ = "Μαρία"  
ΤΕΛΟΣ_ΣΤΑΘΕΡΩΝ
```



Χρησιμοποιούμε το όνομα (π.χ. ΠΟΣΟΣΤΟ) αντί της τιμής (24) όπου θέλουμε μέσα στο πρόγραμμα.

4 ΜΕΤΑΒΛΗΤΕΣ

Θέσεις μνήμης στις οποίες αποθηκεύουμε δεδομένα που μπορεί να μεταβάλλονται κατά τη διάρκεια εκτέλεσης.



Δήλωση μεταβλητών

Συντακτικά οι προγραμματιστές δηλώνουμε τον τύπο και το όνομα της μεταβλητής.

Σύνταξη:

```
ΜΕΤΑΒΛΗΤΕΣ  
<τύπος> <όνομα1>, <όνομα2>, ...  
ΤΕΛΟΣ_ΜΕΤΑΒΛΗΤΩΝ
```

Παράδειγμα:

```
ΜΕΤΑΒΛΗΤΕΣ  
ΑΚΕΡΑΙΕΣ: i, n  
ΠΡΑΓΜΑΤΙΚΕΣ: x, y  
ΧΑΡΑΚΤΗΡΕΣ: ch  
ΛΟΓΙΚΕΣ: flag  
ΤΕΛΟΣ_ΜΕΤΑΒΛΗΤΩΝ
```

5 ΤΕΛΕΣΤΕΣ - ΣΥΝΑΡΤΗΣΕΙΣ - ΕΚΦΡΑΣΕΙΣ

Α. Τελεστές

Σύμβολα που δηλώνουν πράξεις.

Αριθμητικοί

+ - * / ^ (τετραγωνικό)

Σχεσιακοί

< <> < > <= >=

Λογικοί

ΚΑΙ Η ΟΧΙ

Β. Συναρτήσεις

Ειδικές πράξεις που εκτελούνται από τη ΓΛΩΣΣΑ.

Συνάρτηση	Περιγραφή
A_T(x)	Απόλυτη τιμή
HM(x)	Ημίτονο
SYN(x)	Συνημίτονο
ΕΦ(x)	Εφαπτομένη
T_P(x)	Τετραγωνική ρίζα
ΛΟΓ(x)	Λογάριθμος

Γ. Εκφράσεις

Συνδυασμοί τελεστών, σταθερών, μεταβλητών και συναρτήσεων που δίνουν μια τιμή.

Παραδείγματα εκφράσεων:

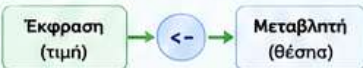
- $5 + 3 * x$
- $A_T(y - 2)$
- $HM(x) + 3.5$
- $(a + b) / 2$



Η σειρά εκτέλεσης καθορίζεται από παρενθέσεις και προτεραιότητα τελεστών.

6 ΕΝΤΟΛΗ ΕΚΧΩΡΗΣΗΣ

Μεταφέρει την τιμή μιας έκφρασης σε μια μεταβλητή.



Σύνταξη:

```
<μεταβλητή> <- <έκφραση>
```

Παραδείγματα:

- $x \leftarrow 5 + 3$
- $y \leftarrow A_T(x)$
- $flag \leftarrow ΑΛΗΘΗΣ$

Η δεξιά πλευρά υπολογίζεται και η τιμή εκχωρείται στη μεταβλητή της αριστερής πλευράς.

7 ΕΝΤΟΛΕΣ ΕΙΣΟΔΟΥ - ΕΞΟΔΟΥ

Α. Είσοδος (INPUT)

Διαβάζουμε δεδομένα από το πληκτρολόγιο.

Σύνταξη:

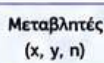
```
ΔΙΑΒΑΣΕ <λίστα μεταβλητών>
```

Παράδειγμα:

```
ΔΙΑΒΑΣΕ x, y, n
```



Πληκτρολόγιο



Μεταβλητές
(x, y, n)

Β. Έξοδος (OUTPUT)

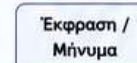
Εμφανίζουμε αποτελέσματα στην οθόνη.

Σύνταξη:

```
ΓΡΑΨΕ <λίστα εκφράσεων ή μηνυμάτων>
```

Παράδειγμα:

```
ΓΡΑΨΕ "Αθροισμα = ", s
```



Οθόνη

8 ΔΟΜΗ ΠΡΟΓΡΑΜΜΑΤΟΣ

Κάθε πρόγραμμα περιέχει κατά σειρά τα παρακάτω τμήματα:

- ✓ ΣΤΑΘΕΡΕΣ (παραμετρικό τμήμα)
- ✓ ΜΕΤΑΒΛΗΤΕΣ
- ✓ ΑΡΧΗ
- ✓ Εντολές (σειριακά ή δομημένα)
- ✓ ΤΕΛΟΣ

Γενικό σχήμα:

```
ΠΡΟΓΡΑΜΜΑ το_όνομα_μου  
ΣΤΑΘΕΡΕΣ  
...  
ΜΕΤΑΒΛΗΤΕΣ  
...  
ΑΡΧΗ  
! Εντολές του προγράμματος  
ΤΕΛΟΣ το_όνομα_μου
```



Πρέπει να δηλώνουμε σταθερές και μεταβλητές πριν από τις εντολές που τις χρησιμοποιούν.



Στόχος κάθε προγράμματος: σωστή λύση, με ταχύτητα και οικονομία σε υπολογιστικούς πόρους.



Ταχύτητα



Οικονομία πόρων

ΚΡΑΤΗΣΕ ΓΙΑ ΤΗ ΜΝΗΜΗ!



Το αλφάβητο περιέχει γράμματα, ψηφία και ειδικούς χαρακτήρες.



Οι τύποι δεδομένων ορίζουν το είδος τιμών, ροδρδδδν νν πνρμμνν.



Σταθερές δεν αλλάζουν. Μεταβλητές αλλάζουν τιμή.



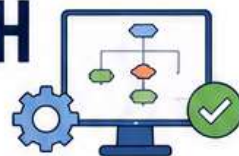
Τελεστές + συναρτήσεις = εκφράσεις που δίνουν τιμή.



Η εκχώρηση (=) αποθηκεύει τιμή σε μεταβλητή.



Είσοδος: ΔΙΑΒΑΣΕ
Έξοδος: ΓΡΑΨΕ
ΑΡΧΗ... ΑΡΧΗ... ΤΕΛΟΣ



ΤΙ ΕΙΝΑΙ;

Οι δομές επιλογής και επανάληψης ελέγχουν τη ροή εκτέλεσης. Ανάλογα με τη συνθήκη (ΑΛΗΘΗΣ ή ΨΕΥΔΗΣ) ή τις ανάγκες επαναλήψεων, εκτελούνται διαφορετικές εντολές.

ΣΤΟΧΟΙ

- ✓ Να ελέγχουμε συνθήκες.
- ✓ Να επαναλαμβάνουμε ενέργειες μέχρι να ικανοποιηθεί μια συνθήκη ή για συγκεκριμένο αριθμό φορών.

ΛΟΓΙΚΕΣ ΕΚΦΡΑΣΕΙΣ

Χρησιμοποιούμε σταθερές, μεταβλητές, αριθμητικές παραστάσεις, συγκριτικούς και λογικούς τελεστές. Το αποτέλεσμα μιας λογικής έκφρασης είναι μόνο ΑΛΗΘΗΣ ή ΨΕΥΔΗΣ.

✓ ΑΛΗΘΗΣ

✗ ΨΕΥΔΗΣ

1. ΕΝΤΟΛΕΣ ΕΠΙΛΟΓΗΣ

Ελέγχουμε μια συνθήκη και εκτελούμε εντολές ανάλογα με το αποτέλεσμα.

1.1 ΑΠΛΗ ΕΠΙΛΟΓΗ (ΑΝ ... ΤΟΤΕ)

Αν η συνθήκη είναι ΑΛΗΘΗΣ, εκτελούνται οι εντολές.



Σύνταξη

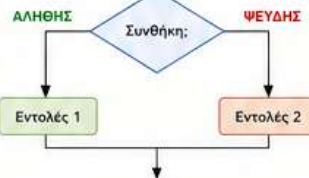
```
ΑΝ συνθήκη ΤΟΤΕ
  εντολή-1
  εντολή-2
  ...
  εντολή-n
ΤΕΛΟΣ_ΑΝ
```

Παράδειγμα

```
ΑΝ Βάρος < 80 ΤΟΤΕ
  ΓΡΑΨΕ 'Ελαφρύς, κοντός'
ΤΕΛΟΣ_ΑΝ
```

1.2 ΑΠΛΗ ΕΠΙΛΟΓΗ (ΑΝ ... ΤΟΤΕ ... ΑΛΛΙΩΣ)

Αν η συνθήκη είναι ΑΛΗΘΗΣ εκτελείται το πρώτο τμήμα, διαφορετικά το δεύτερο.



Σύνταξη

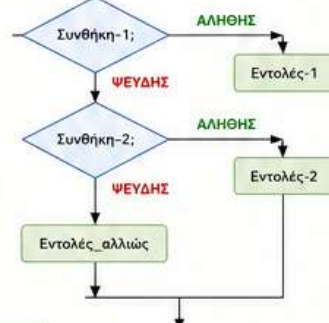
```
ΑΝ συνθήκη ΤΟΤΕ
  εντολές-1
ΑΛΛΙΩΣ
  εντολές-2
ΤΕΛΟΣ_ΑΝ
```

Παράδειγμα

```
ΑΝ Βάρος < 80 ΚΑΙ Ύψος < 1.70 ΤΟΤΕ
  ΓΡΑΨΕ 'Ελαφρύς, κοντός'
ΑΛΛΙΩΣ
  ΓΡΑΨΕ 'Όχι ελαφρύς, όχι κοντός'
ΤΕΛΟΣ_ΑΝ
```

1.3 ΠΟΛΛΑΠΛΗ ΕΠΙΛΟΓΗ (ΑΝ ... ΤΟΤΕ ... ΑΛΛΙΩΣ_ΑΝ ... ΑΛΛΙΩΣ)

Ελέγχει διαδοχικά πολλές συνθήκες.



Σύνταξη

```
ΑΝ συνθήκη-1 ΤΟΤΕ
  εντολές-1
ΑΛΛΙΩΣ_ΑΝ συνθήκη-2 ΤΟΤΕ
  εντολές-2
  ...
ΑΛΛΙΩΣ
  εντολές-n
ΤΕΛΟΣ_ΑΝ
```

Παράδειγμα

```
ΑΝ Βάρος < 80 ΚΑΙ Ύψος < 1.70 ΤΟΤΕ
  ΓΡΑΨΕ 'Ελαφρύς, κοντός'
ΑΛΛΙΩΣ_ΑΝ Βάρος < 80 ΤΟΤΕ
  ΓΡΑΨΕ 'Ελαφρύς, όχι κοντός'
ΑΛΛΙΩΣ
  ΓΡΑΨΕ 'Όχι ελαφρύς'
ΤΕΛΟΣ_ΑΝ
```

Η χρήση εμφωλευμένων εντολών επιτρέπει πολύπλοκες δομές ελέγχου. Οι συνθήκες μπορούν να συνδυάζονται με λογικούς τελεστές: ΚΑΙ (Λ), Ή (V), ΟΧΙ (-).

2. ΕΝΤΟΛΕΣ ΕΠΑΝΑΛΗΨΗΣ

Επαναλαμβάνουμε ενέργειες όσο μια συνθήκη ικανοποιείται ή για καθορισμένο αριθμό επαναλήψεων.

2.1 ΟΣΟ ... ΕΠΑΝΑΛΑΒΕ

Ελέγχει τη συνθήκη στην αρχή. Όσο είναι ΑΛΗΘΗΣ εκτελεί τις εντολές.



Σύνταξη

```
ΟΣΟ συνθήκη ΕΠΑΝΑΛΑΒΕ
  εντολή-1
  εντολή-2
  ...
  εντολή-n
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
```

Παράδειγμα

```
ΟΣΟ X > 0 ΕΠΑΝΑΛΑΒΕ
  ΓΡΑΨΕ X
  X <- X - 1
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
```

Η εκτέλεση μπορεί να γίνει 0, 1 ή περισσότερες φορές.

2.2 ΜΕΧΡΙΣ_ΟΤΟΥ

Ελέγχει τη συνθήκη στο τέλος. Εκτελεί τις εντολές τουλάχιστον μία φορά.



Σύνταξη

```
ΜΕΧΡΙΣ_ΟΤΟΥ συνθήκη
  εντολή-1
  εντολή-2
  ...
  εντολή-n
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
```

Παράδειγμα

```
ΜΕΧΡΙΣ_ΟΤΟΥ X <= 0
  ΓΡΑΨΕ 'Δώσε θετικό αριθμό:'
  ΔΙΑΒΑΣΕ X
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
```

Η εκτέλεση γίνεται οπωσδήποτε τουλάχιστον μία φορά.

2.3 ΓΙΑ ... ΑΠΟ ... ΜΕΧΡΙ ... ΜΕ_ΒΗΜΑ

Εκτελεί τις εντολές για συγκεκριμένο αριθμό επαναλήψεων.



Σύνταξη

```
ΓΙΑ μετρητή ΑΠΟ αρχική_τιμή ΜΕΧΡΙ τελική_τιμή ΜΕ_ΒΗΜΑ βήμα
  εντολή-1
  εντολή-2
  ...
  εντολή-n
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
```

Παράδειγμα

```
ΓΙΑ I ΑΠΟ 1 ΜΕΧΡΙ 5 ΜΕ_ΒΗΜΑ 1
  ΓΡΑΨΕ I
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
```

Ο μετρητής παίρνει τιμές: αρχική, αρχική+βήμα, ... μέχρι να ξεπεράσει την τελική τιμή.

Η ΕΝΤΟΛΗ ΕΠΙΛΕΞΕ

Η εντολή ΕΠΙΛΕΞΕ επιτρέπει την επιλογή μιας από πολλές περιπτώσεις τιμών.

Γενική μορφή:

```
ΕΠΙΛΕΞΕ <έκφραση>
  ΠΕΡΙΠΤΩΣΗ <λίστα_τιμών_1>
    <εντολές_1>
  ΠΕΡΙΠΤΩΣΗ <λίστα_τιμών_2>
    <εντολές_2>
  ...
  ΠΕΡΙΠΤΩΣΗ ΑΛΛΙΩΣ
    <εντολές_αλλιώς>
ΤΕΛΟΣ_ΕΠΙΛΟΓΗΣ
```



ΛΙΣΤΑ ΤΙΜΩΝ (ΤΕΛΕΣΤΕΣ)

Τελεστής	Περιγραφή	Παράδειγμα
=	Ισότητα	Αριθμός = 0
<>	Ανισότητα	Όνομα <> 'Κώστας'
>	Μεγαλύτερο από	Τιμή > 10000
>=	Μεγαλύτερο ή ίσο	$X+Y \geq (A+B)/\Gamma$
<	Μικρότερο από	$B*2-4*A*\Gamma < 0$
<=	Μικρότερο ή ίσο	Βάρος <= 500

ΛΟΓΙΚΟΙ ΤΕΛΕΣΤΕΣ

ΚΑΙ (Λ) : ΑΛΗΘΗΣ μόνο αν και τα δύο είναι ΑΛΗΘΗΣ
Ή (V) : ΑΛΗΘΗΣ αν τουλάχιστον ένα είναι ΑΛΗΘΗΣ
ΟΧΙ (-) : Αντιστροφή της τιμής (ΑΛΗΘΗΣ ↔ ΨΕΥΔΗΣ)

ΣΥΚΝΕΣ ΧΡΗΣΙΜΕΣ ΥΠΕΝΘΥΜΙΣΕΙΣ

- Σε κάθε δομή επιλογής και επανάληψης η ροή εκτέλεσης ακολουθεί τα βέλη του διαγράμματος ροής.
- Οι όροι συνθήκης είναι λογικές εκφράσεις που δίνουν ΑΛΗΘΗΣ ή ΨΕΥΔΗΣ.
- Προσοχή σε απεριόριστες επαναλήψεις!
- Επιλέγουμε την κατάλληλη δομή για κάθε περίπτωση.



ΚΑΛΕΣ ΠΡΑΚΤΙΚΕΣ

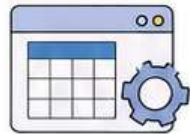
- ✓ Διαβάζουμε προσεκτικά το πρόβλημα.
- ✓ Σχεδιάζουμε πρώτα το διάγραμμα ροής.
- ✓ Χρησιμοποιούμε σαφείς ονόματα μεταβλητών.
- ✓ Ελέγχουμε την ορθότητα με δοκιμές.





ΠΙΝΑΚΕΣ

Κεφάλαιο 9 • Θεωρία για αποστήθιση



ΕΙΣΑΓΩΓΗ

Οι πίνακες χρησιμοποιούνται για διαχείριση μεγάλου αριθμού δεδομένων ίδιου τύπου και στο κεφάλαιο παρουσιάζονται μονοδιάστατοι, πολυδιάστατοι πίνακες και βασικές επεξεργασίες τους.



ΣΤΟΧΟΙ

- Να επιλέγει το είδος του πίνακα.
- Να ορίζει πίνακες σε πρόγραμμα.
- Να εισάγει, να επεξεργάζεται και να εκτυπώνει στοιχεία πίνακα.
- Να αποφασίζει αν χρειάζεται πίνακας.
- Να αναφέρει βασικές επεξεργασίες.
- Να αναζητά και να ταξινομεί στοιχεία.

ΓΙΑΤΙ ΧΡΕΙΑΖΟΜΑΣΤΕ ΠΙΝΑΚΑ;

Θέλουμε να διαβάσουμε τις θερμοκρασίες των 30 ημερών ενός μήνα.

- Μια μεταβλητή: Θερμοκρασία → αρκεί για τη μέση τιμή.
- Αν μετά θέλουμε να βρούμε πόσες ημέρες ήταν κάτω από τη μέση, πρέπει να έχουν διατηρηθεί όλες οι τιμές.

Άρα καλύτερη λύση: πίνακας Θερμοκρασία[30].

30 χωριστές μεταβλητές



1 πίνακας με 30 στοιχεία



ΣΗΜΑΝΤΙΚΗ ΥΠΕΝΘΥΜΙΣΗ

Η ανάγνωση, η επεξεργασία και η εκτύπωση των στοιχείων των πινάκων γίνεται πάντοτε από βρόχους ... υλοποιούνται καλύτερα με την εντολή επανάληψης **ΓΙΑ**.

9.1 ΜΟΝΟΔΙΑΣΤΑΤΟΙ ΠΙΝΑΚΕΣ

ΟΡΙΣΜΟΣ

Πίνακας είναι ένα σύνολο αντικειμένων ίδιου τύπου, τα οποία αναφέρονται με ένα κοινό όνομα. Κάθε ένα από τα αντικείμενα που απαρτίζουν τον πίνακα λέγεται **στοιχείο** του πίνακα. Η αναφορά σε ατομικά στοιχεία του πίνακα γίνεται με το όνομα του πίνακα ακολουθούμενο από ένα δείκτη.

Οι πίνακες που χρησιμοποιούν ένα μόνο δείκτη ονομάζονται **μονοδιάστατοι πίνακες**.

ΔΗΛΩΣΗ ΠΙΝΑΚΑ (παράδειγμα)

ΜΕΤΑΒΛΗΤΕΣ

ΠΡΑΓΜΑΤΙΚΕΣ: Θερμοκρασία[30]

ΑΝΑΠΑΡΑΣΤΑΣΗ ΣΤΗ ΜΝΗΜΗ (παράδειγμα)

Πίνακας Θερμοκρασία με 30 στοιχεία

Διεύθυνση μνήμης
1000
1004
1008
...
1112
1116

Θερμοκρασία[1]	26
Θερμοκρασία[2]	27
Θερμοκρασία[3]	24
...	...
Θερμοκρασία[29]	21
Θερμοκρασία[30]	25

→ Θερμοκρασία[1] = 26
→ Θερμοκρασία[2] = 27
→ Θερμοκρασία[3] = 24
...
→ Θερμοκρασία[29] = 21
→ Θερμοκρασία[30] = 25

Κοινό όνομα
(για όλα τα στοιχεία)

Δείκτης
(δείχνει το στοιχείο)

Τιμή
(στοιχείο πίνακα)

9.2 ΠΟΤΕ ΠΡΕΠΕΙ ΝΑ ΧΡΗΣΙΜΟΠΟΙΟΥΝΤΑΙ ΠΙΝΑΚΕΣ

✓ ΧΡΗΣΙΜΟΙ / ΑΠΑΡΑΙΤΗΤΟΙ

Όταν τα δεδομένα πρέπει να διατηρούνται στη μνήμη μέχρι το τέλος της εκτέλεσης του προγράμματος.

Πολύ σημαντικό παράδειγμα

Για τη μέση τιμή και την τυπική απόκλιση δεν είναι απαραίτητο να διατηρούνται όλες οι τιμές, αρκεί να διατηρούμε βοηθητικές μεταβλητές. Όμως, για τη διάμεσο χρειάζεται ταξινόμηση, άρα πρέπει να διατηρηθούν όλες οι τιμές → χρειάζεται πίνακας.

ΣΥΜΠΕΡΑΣΜΑ: Αν τα δεδομένα δεν χρειάζεται να παραμείνουν στη μνήμη, η χρήση πίνακα μπορεί να αποφεύγεται.

✗ ΜΕΙΟΝΕΚΤΗΜΑΤΑ

- Απαιτούν μνήμη.
- Δεσμεύουν θέσεις μνήμης από την αρχή.
- Είναι στατικές δομές και το μέγεθός τους παραμένει σταθερό.
- Άρα μπορεί να περιορίζουν τις δυνατότητες του προγράμματος.

9.3 ΠΟΛΥΔΙΑΣΤΑΤΟΙ ΠΙΝΑΚΕΣ

ΠΑΡΑΔΕΙΓΜΑ: Θερμοκρασίες 30 ημερών x 10 πόλεων

Ημέρες (γραμμές)	Πόλεις (στήλες)					
	1	2	3	...	10	
1	27	25	26	...	24	
2	28	26	25	...	23	
3	24	22	23	...	21	
...	...	...	...	...	...	
30	21	20	22	...	19	

Παράδειγμα αναφοράς στοιχείου:
Θερμοκρασία[30,1] = 27

- Πρώτος δείκτης → γραμμή / ημέρα (1 έως 30).
- Δεύτερος δείκτης → στήλη / πόλη (1 έως 10).

ΔΗΛΩΣΕΙΣ (παράδειγμα)

ΜΕΤΑΒΛΗΤΕΣ

ΠΡΑΓΜΑΤΙΚΕΣ: Θερμοκρασία[30,10], Μέση[10]

ΑΝΑΓΝΩΣΗ / ΕΠΕΞΕΡΓΑΣΙΑ ΜΕ ΕΜΦΩΛΕΥΜΕΝΕΣ ΕΝΤΟΛΕΣ ΓΙΑ

ΓΙΑ i ΑΠΟ 1 ΜΕΧΡΙ 30
ΓΙΑ j ΑΠΟ 1 ΜΕΧΡΙ 10
... επεξεργασία στοιχείου
Θερμοκρασία[i, j] ...
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

★ Υπάρχουν και τριδιάστατοι ή πολυδιάστατοι πίνακες, αλλά τα περισσότερα προβλήματα αντιμετωπίζονται με μονοδιάστατους ή διδιάστατους.

ΠΑΡΑΔΕΙΓΜΑ 4 – ΚΙΝΗΜΑΤΟΓΡΑΦΟΙ

Πίνακας 1: Ονόματα[10]

Πίνακας 2: Εισπράξεις[10,7] (σε ευρώ)

1	«ΑΣΤΕΡΑΣ»
2	«ΠΑΛΑΣ»
3	«ΑΠΟΛΛΩΝ»
...	...
10	«ΑΛΚΥΩΝ»

Κινηματογράφοι
(γραμμές)

	Ημέρες (στήλες)						
	1	2	3	4	5	6	7
1	...	...	...	...	...	...	...
2	...	...	...	...	...	...	...
3	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...
10	...	...	...	...	...	...	...

Τμήμα Α: Αθροίσματα ανά γραμμή (κάθε κινηματογράφο)

1. Για κάθε γραμμή (κινηματογράφο) υπολογίζει το άθροισμα των 7 ημερών.
2. Βρίσκει τον κινηματογράφο με τη μέγιστη συνολική εισπράξη.

ΓΙΑ i ΑΠΟ 1 ΜΕΧΡΙ 10
άθροισμα = 0
ΓΙΑ j ΑΠΟ 1 ΜΕΧΡΙ 7
άθροισμα + Εισπράξεις[i, j]
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
σύγκριση για μέγιστο άθροισμα
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

Τμήμα Β: Αθροίσματα ανά στήλη (κάθε ημέρα)

1. Για κάθε στήλη (ημέρα) υπολογίζει το άθροισμα των 10 κινηματογράφων.
2. Βρίσκει την ημέρα με τη μέγιστη συνολική εισπράξη.

ΓΙΑ j ΑΠΟ 1 ΜΕΧΡΙ 7
άθροισμα = 0
ΓΙΑ i ΑΠΟ 1 ΜΕΧΡΙ 10
άθροισμα + Εισπράξεις[i, j]
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
σύγκριση για μέγιστο άθροισμα
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

Προσέξτε:
ιδίος εμφωλευμένος βρόχος, αλλά με διαφορετική σειρά (γραμμές/στήλες).

9.4 ΤΥΠΙΚΕΣ ΕΠΕΞΕΡΓΑΣΙΕΣ ΠΙΝΑΚΩΝ

1

Υπολογισμός αθροισμάτων στοιχείων του πίνακα (ανά γραμμή ή ανά στήλη)

Υπολογίζουμε το άθροισμα όλων των στοιχείων του πίνακα ή το άθροισμα ανά γραμμή (σε διδιάστατο πίνακα) ή ανά στήλη.

2

Εύρεση μέγιστου ή ελάχιστου στοιχείου



Εντοπίζουμε τη μεγαλύτερη (μέγιστη) ή τη μικρότερη (ελάχιστη) τιμή του πίνακα.

3

Ταξινόμηση των στοιχείων του πίνακα



Ταξινομούμε τα στοιχεία σε αύξουσα ή φθίνουσα σειρά. Η μέθοδος της ευθείας ανταλλαγής είναι απλή αλλά όχι η πιο αποδοτική.

4

Αναζήτηση ενός στοιχείου του πίνακα



Σειριακή αναζήτηση → χρησιμοποιείται υποχρεωτικά σε μη ταξινομημένους πίνακες.
Διαδική αναζήτηση → μόνο σε ταξινομημένους πίνακες και είναι πιο αποδοτική.

5

Συγχώνευση δύο πινάκων



Δημιουργείται από δύο ή περισσότερους ταξινομημένους πίνακες ένας άλλος επίσης ταξινομημένος.

ΑΝΑΚΕΦΑΛΑΙΩΣΗ / ΚΡΑΤΗΣΕ ΓΙΑ ΤΗ ΜΝΗΜΗ



1 Πίνακας = ομάδα μεταβλητών ίδιου τύπου με κοινό όνομα.



2 Στοιχείο πίνακα = όνομα + δείκτης.

A[i]

3 Οι πίνακες αποθηκεύονται σε διαδοχικές θέσεις μνήμης.



4 Μπορεί να είναι μονοδιάστατοι, διδιάστατοι ή πολυδιάστατοι.



5 Η επεξεργασία τους γίνεται συνήθως με εντολές **ΓΙΑ**.



6 Βασικές επεξεργασίες: αναζήτηση, ταξινόμηση, συγχώνευση.





ΚΕΦΑΛΑΙΟ 10 — ΥΠΟΠΡΟΓΡΑΜΜΑΤΑ



ΑΕΠΠ • Αποστήθιση Θεωρίας / Γρήγορη Επανάληψη

1 Τμηματικός προγραμματισμός



Ορισμός: Η τεχνική σχεδίασης και ανάπτυξης ενός προγράμματος ως σύνολο από απλούστερα τμήματα.



Βασική ιδέα: Διάσπαση σύνθετου προβλήματος σε μικρότερα υποπροβλήματα.



Λέξεις-κλειδιά: top-down προσέγγιση, ανάλυση, επιμέρους τμήματα.

4 Πλεονεκτήματα



Διευκολύνουν την ανάπτυξη του αλγορίθμου και του προγράμματος.



Κάνουν ευκολότερη την κατανόηση και τη διόρθωση.



Μειώνουν χρόνο συγγραφής και πιθανότητες λάθους.



Επιτρέπουν επαναχρησιμοποίηση και επεκτείνουν τις δυνατότητες της γλώσσας.

2 Τι είναι το υποπρόγραμμα;



- Υποπρόγραμμα είναι ένα αυτόνομο τμήμα προγράμματος που εκτελεί συγκεκριμένο έργο.
- Χρησιμοποιείται για οργάνωση, επαναχρησιμοποίηση και καλύτερο έλεγχο.

3 Χαρακτηριστικά υποπρογραμμάτων

1



Μία είσοδος – μία έξοδος

2



Σχετική ανεξαρτησία από τα άλλα υποπρογράμματα

3



Μικρό μέγεθος και μία σαφής λειτουργία

5 Παράμετροι

- Παράμετρος:** μεταβλητή μέσω της οποίας περνούν τιμές από ένα τμήμα προγράμματος σε άλλο.
- Τα υποπρογράμματα επικοινωνούν με το κύριο πρόγραμμα μέσω παραμέτρων.

Τυπικές παράμετροι

στη δήλωση του υποπρογράμματος



Πραγματικές παράμετροι

στην κλήση του υποπρογράμματος

Κανόνες παραμέτρων

- ✓ Ίδιο πλήθος
- ✓ Αντιστοίχιση κατά θέση
- ✓ Ίδιος τύπος

6 Διαδικασία ή Συνάρτηση;

ΔΙΑΔΙΚΑΣΙΑ

- Μπορεί να εκτελεί όλες τις λειτουργίες ενός προγράμματος
- Μπορεί να διαβάσει, να υπολογίζει, να μεταβάλλει τιμές, να εκτυπώνει
- Καλείται με:
«ΚΑΛΕΣΕ όνομα(παραμέτροι)»

ΣΥΝΑΡΤΗΣΗ

- Υπολογίζει και επιστρέφει μόνο μία τιμή
- Η τιμή μπορεί να είναι αριθμητική, λογική ή χαρακτήρας
- Χρησιμοποιείται μέσα σε έκφραση

★ Διαδικασία = ενέργειες • Συνάρτηση = μία τιμή ★

7 Γενική μορφή

A Συνάρτηση

ΣΥΝΑΡΤΗΣΗ Όνομα(λίστα_παραμέτρων): Τύπος
Μεταβλητές
ΑΡΧΗ
Όνομα <- έκφραση
ΤΕΛΟΣ_ΣΥΝΑΡΤΗΣΗΣ

B Διαδικασία

ΔΙΑΔΙΚΑΣΙΑ Όνομα(λίστα_παραμέτρων)
Μεταβλητές
ΑΡΧΗ
εντολές
ΤΕΛΟΣ_ΔΙΑΔΙΚΑΣΙΑΣ

8 Εμβέλεια μεταβλητών – σταθερών

- Εμβέλεια** = το τμήμα του προγράμματος στο οποίο ισχύει μια μεταβλητή.
- Στη ΓΛΩΣΣΑ έχουμε περιορισμένη εμβέλεια.
- Οι μεταβλητές και οι σταθερές είναι τοπικές στο πρόγραμμα ή στο υποπρόγραμμα όπου δηλώνονται.
- Η μεταφορά τιμών γίνεται μέσω παραμέτρων.

! Ίδιο όνομα σε διαφορετικά τμήματα ≠ ίδια μεταβλητή

9 Αναδρομή

- Αναδρομή:** η δυνατότητα ενός υποπρογράμματος να καλεί τον εαυτό του.
- Κάθε αναδρομικός ορισμός έχει:
 - τιμή βάσης / συνθήκη τερματισμού
 - αναδρομική σχέση
- Πλεονέκτημα:** συχνά πιο φυσική διατύπωση.
- Μειονέκτημα:** συχνά μεγαλύτερος χρόνος εκτέλεσης.

Παραγοντικό:

Αν $N = 0$ τότε 1

Αλλιώς

$N * \text{Παραγοντικό}(N-1)$



10 SUPER SOS για αποστήθιση



1 Υποπρόγραμμα = αυτόνομο τμήμα προγράμματος



2 Δύο είδη: διαδικασίες και συναρτήσεις



3 Οι διαδικασίες καλούνται με ΚΑΛΕΣΕ

$f(x)$

4 Οι συναρτήσεις επιστρέφουν μία τιμή



5 Η επικοινωνία γίνεται με παραμέτρους



6 Στη ΓΛΩΣΣΑ η εμβέλεια είναι περιορισμένη

